Methodology: Data Collection (Draft)

Working draft, July 15, 2026. Rough prose on purpose. Items in [BRACKETS] need confirmation before this section is final.

1. Repository mining

We collected candidate repositories using the SEART GitHub Search tool. The query was scoped to repositories with Python as the main language, created after August 31, 2025 [CONFIRM: any additional SEART criteria set at collection time, such as minimum stars, minimum commits, non-fork only, or excluding archived repositories]. The export contained 15,739 candidate repositories.

The creation-date scoping serves two purposes. First, it restricts the dataset to code written recently, reflecting current MLflow usage practices. Second, it provides the contamination guarantee described in Section 3.

2. License filtering

Because repository contents are used as study material, we retained only repositories under permissive open-source licenses: MIT, Apache-2.0, BSD-2-Clause, and BSD-3-Clause. This reduced the candidate set from 15,739 to 9,154 repositories (58.2 percent retained). The license distribution of the retained set was dominated by MIT (6,628) and Apache-2.0 (2,526).

3. Contamination control

The study evaluates Qwen 3 8B (released April 29, 2025) and Llama 3.1 8B (released July 23, 2024). To ensure that no evaluated code could have appeared in either model's training data, all repositories must postdate the later release. The effective cutoff is therefore April 29, 2025.

We verified the collected set against this cutoff using repository creation dates: the earliest creation date in the dataset is August 31, 2025, which postdates the cutoff by four months. The contamination constraint is thus satisfied by construction, since the SEART query was already scoped by creation date. We use creation date rather than last commit date as the conservative choice: a repository created after a model's release cannot have contributed any content to its training corpus, regardless of commit history. [Confirmed with mentor: creation date is the agreed field.]

4. MLflow dependency pre-filter

SEART exposes repository metadata only, so a two-stage process was used to identify genuine MLflow usage. The first stage scans each candidate repository's root-level manifest files (requirements.txt, requirements-dev.txt, pyproject.toml, setup.py, setup.cfg, environment.yml, environment.yaml, Pipfile, conda.yaml) for a declaration of mlflow, using the GitHub REST API. Of the 9,154 licensed candidates, 52 repositories (0.6 percent) declared mlflow as a dependency.

A known limitation of this stage is that only root-level manifests are scanned; repositories that declare dependencies in subdirectories, or that use mlflow without declaring it in a manifest, are excluded. [NOTE:

widening decision pending discussion with supervisors; update this paragraph if the collection criteria change.]

5. MLflow file detection

The second stage identifies the specific Python files within each pre-filtered repository that use the MLflow API. For each repository, the GitHub code search API retrieves Python files mentioning mlflow; each file is then parsed with Python's ast module. A file is recorded when it (a) imports mlflow or an mlflow submodule, or (b) contains direct call expressions on the mlflow module name, such as mlflow.log_param(...). For each recorded file we store the import flag and the count of direct API calls. Files that failed AST parsing fell back to a textual check.

Of the 52 pre-filtered repositories, 33 contained at least one detectable MLflow Python file; the remaining 19 declared the dependency without detectable usage in Python files. The final dataset comprises 105 unique MLflow Python files across 33 repositories, after removal of 7 duplicate records introduced by a resumed collection run.

6. Pipeline validation

Following supervisor guidance, we validated the pipeline with a manual audit. Ten repositories were sampled uniformly at random from the 33-repository final set using a fixed seed (42) for reproducibility. For each sampled repository we manually verified, via GitHub's code search and file inspection: (a) that the repository genuinely uses MLflow, (b) that the pipeline identified the correct set of files, including checking that files the pipeline excluded were correct exclusions, and (c) that the recorded call counts were accurate, with several files verified line by line.

All ten repositories matched the pipeline output (10 of 10), with several exact call-count matches (for example, 4 of 4, 8 of 8, and 15 of 15 in three repositories).

7. Observed patterns relevant to detection and the failure taxonomy

The manual validation surfaced four recurring code patterns that complicate MLflow usage detection in real-world repositories:

1. Lazy or defensive imports. In 8 of the 10 sampled repositories, mlflow is imported inside try/except blocks, inside functions, or behind availability flags (for example _HAS_MLFLOW or _MLFLOW_AVAILABLE), so that the host application works when MLflow is not installed. Several repositories implement no-op fallbacks such as a NoOpTracker class or a pass-through trace decorator.
2. Module stored as an object attribute. One repository imports mlflow lazily and stores the module on an instance (self._mlflow = mlflow); all subsequent usage occurs through attribute access, so the file registers zero direct module calls despite substantial MLflow usage.

3. Name shadowing by local variables. One repository assigns its own client object to a local variable named mlflow; call counting on the bare name then includes instance-method calls on the shadowing variable.
4. Mock objects named mlflow in tests. One repository's test files create MagicMock objects named mlflow and assert on them (mlflow.log_metrics.assert_called_once()); these register as calls without any import, and are test scaffolding rather than API usage.

The AST-based detector behaved accurately under all four patterns in the sampled set. These observations inform both the interpretation of per-file call counts and the design of the planned failure taxonomy: generated code may legitimately express MLflow usage through patterns 1 and 2, and evaluation tooling must not misattribute patterns 3 and 4.

8. Summary of the collection funnel

Stage	Count
SEART export (Python, created after 2025-08-31)	15,739
After license filter (MIT, Apache-2.0, BSD-2, BSD-3)	9,154
After contamination date check (cutoff 2025-04-29; 0 removed by construction)	9,154
After MLflow manifest pre-filter	52
Repositories with detected MLflow Python files	33
Total MLflow Python files (deduplicated)	105

TODO before this section is final

- Fill the [BRACKETS] above (exact SEART query parameters).
- Update Section 4 if the widening decision changes the collection criteria.
- Add citations: SEART GHS tool paper; GitHub-mining methodology papers used to justify the filtering criteria (from the May criteria document).
- Decide where the validation spreadsheet lives (appendix or supplementary material).